

Keycloak Admin Console — Complete Beginner's Guide

Every Sidebar Item Explained

Version: Keycloak 26.x **Our admin URL:** <https://keycloak-production-4f96.up.railway.app/admin>
Login credentials: [superadmin](#) / [Admin@123](#)

Contents

1. [Realm Dropdown \(Top-Left\)](#)
 2. [Clients](#)
 3. [Client Scopes](#)
 4. [Realm Roles](#)
 5. [Users](#)
 6. [Groups](#)
 7. [Sessions](#)
 8. [Events](#)
 9. [Identity Providers](#)
 10. [User Federation](#)
 11. [Authentication](#)
 12. [Security Defenses](#)
 13. [Server Info](#)
 14. [Realms \(Advanced\)](#)
 15. [Quick Reference — Common Tasks](#)
 16. [Troubleshooting Common Issues](#)
-

1. Realm Dropdown (Top-Left)

Where it is

Top-left corner of the admin console. It shows the current realm name (e.g., [govasset](#)).

What it does

Keycloak groups everything into **realms**. A realm is like a separate universe — users, clients, roles in one realm cannot see anything in another realm. Think of a realm as a **completely separate organization**.

The two realms you always have

Realm	Purpose
master	The admin realm. Only admins use this. It manages the Keycloak server itself.

Realm	Purpose
govasset	The main realm. Our application and external systems (Group 2, etc.) use this. Regular users live here.

What we do here

- **Switch between `master` and `govasset`** depending on what we need to configure
- Most day-to-day work happens in the **govasset** realm

Common issues

- **"I can't see my client/user"** — You're probably in the wrong realm. Switch to `govasset`.
- **"I gave user admin role but they still can't access admin console"** — The `admin` role must be assigned in the `master` realm, not `govasset`.

2. Clients

What it is

A **client** represents an application or service that connects to Keycloak for authentication. Every system that uses Keycloak needs its own client.

The clients we currently have

Client Name	Type	Who uses it
<code>account</code>	Built-in	The "Manage Account" page users see
<code>account-console</code>	Built-in	The account management UI
<code>admin-cli</code>	Built-in	Used by admin API scripts
<code>broker</code>	Built-in	Used for identity brokering
<code>realm-management</code>	Built-in	Internal Keycloak management
<code>security-admin-console</code>	Built-in	The admin console itself
<code>govasset-django</code>	Ours	Our Django application (SSO)
<code>group2-app</code>	Ours	External system integration

Key settings inside a Client

A) Settings tab:

Setting	What it does	Example
Client ID	Unique name for this client	<code>govasset-django</code>
Client authentication	ON = client needs a secret. OFF = public client (no secret).	ON for server apps, OFF for SPAs

Setting	What it does	Example
Standard flow	Authorization Code flow (recommended)	✓ ON for most apps
Direct access grants	Resource Owner flow (username + password directly)	✓ ON if using Type B (Direct API)
Valid redirect URIs	Where users CAN be redirected after login	https://myapp.com/auth/callback
Valid post logout redirect URIs	Where users CAN be redirected after logout	https://myapp.com/bye
Web origins	Allowed domains for CORS (for SPAs)	https://myapp.com
Root URL	Base URL used for redirects	https://myapp.com

B) Credentials tab:

- **Client Secret** — The secret you share with the external system. Keep it secret.
- **Regenerate** — Creates a new secret (old one stops working immediately).

C) Roles tab:

- Custom roles specific to this client (rarely used in our setup).

What we do here

- **Create a new client** when a new external system wants to integrate
- **Configure redirect URIs** — must match exactly what the external system sends
- **Copy the Client Secret** to give to the external system
- **Turn flows ON/OFF** — e.g., enable Direct Access Grants for Type B (Direct API) integration

Common issues

- **"Invalid redirect URI"** — The URI in the login URL does not match what's in Valid Redirect URIs. Exact match required (trailing slash matters!).
- **"401 Unauthorized" on token exchange** — Client Secret is wrong. Regenerate and update the external system.
- **"403 Forbidden" on account page** — Web Origins is empty for [account-console](#) client (we just fixed this).

3. Client Scopes

What it is

Client scopes define what information (claims) Keycloak includes in tokens. Think of them as **pre-made permission packages**.

Pre-built scopes you always see

- **profile** — User's name, username, picture
- **email** — User's email, verified status
- **roles** — User's realm roles
- **web-origins** — Allowed origins for CORS
- **acr** — Authentication context reference

What we do here

- **Usually nothing** — the defaults work for our setup
- **Add custom attributes** if we need to include extra info in tokens (e.g., **ministry**)

Common issues

- **"Token does not have the info I need"** — The required scope is not assigned to the client. Add it in Client → Client scopes tab.
- **"Token is too large"** — Too many scopes. Only request what you need.

4. Realm Roles

What it is

Roles are permissions. You create a role, then assign it to users. In our system, we map these to our 5 Django roles.

Our roles in govasset realm

Role	Purpose
user	Default role for all users in govasset realm
offline_access	Allows offline tokens (long-lived)
uma_authorization	For user-managed access
default-roles-govasset	Automatically assigned to all users

What we do here

- **Create new roles** if we need custom permissions
- **Assign roles to users** (Users → user → Role mapping)
- **Map roles to clients** so the role appears in the token

The confusing part: **admin** role vs our **SUPER_ADMIN**

- The **admin** role in the **master** realm gives access to the Keycloak admin console ITSELF
- The **SUPER_ADMIN** role in our Django system gives access to the Government Asset Platform features
- They are completely separate. A Django SUPER_ADMIN might not have Keycloak admin access.

Common issues

- **"I deleted the admin role"** — You locked yourself out. Redeploy Keycloak to recreate from env vars.

- **"User has the right role but no access"** — You might have assigned the role in the wrong realm.

5. Users

What it is

A list of all users in the current realm. Each user represents a person who can log in.

What each field means

Field	What it is
Username	Login name (must be unique)
Email	User's email (optional but recommended)
Email verified	Whether they confirmed their email
First name / Last name	Display name
Required user actions	Actions user MUST do on next login (e.g., "Update Password")
Groups	Groups the user belongs to
Role mapping	Roles assigned to this user

What we do here

- **Create users** — Add new government employees
- **Reset passwords** — When a user forgets their password
- **Temporarily disable users** — Set Enabled to OFF
- **Assign roles** — Link user to their role
- **Required user actions** — Force password change on next login

Credentials tab for a user

Action	What it does
Set password	Manually set the user's password
Temporary password	ON = user must change password on next login
OTP policy	Configure 2-factor authentication

What happens when you create a user here

Keycloak only knows about the user's **identity** (username + password). The user still needs to exist in our Django database with a role and ministry assignment to access the platform. That's where our `setup_demo_data` command or admin UI comes in.

Common issues

- **"User exists in Keycloak but can't log in"** — User may not exist in Django. Create them in Django admin too.
 - **"Wrong password"** — Check Caps Lock. Reset if needed.
 - **"User locked out"** — Too many wrong attempts. Go to Security Defenses or temporarily re-enable.
-

6. Groups

What it is

Groups let you assign roles to multiple users at once. When you add a user to a group, they inherit all the roles assigned to that group.

What we do here

- **Rarely used** in our current setup — we assign roles to individual users
- Could be useful in future: create a "Health Ministry Admins" group with the admin role pre-assigned

Common issues

- **"User has weird permissions"** — Check if they inherited roles from a group
 - **Group not showing up** — Groups are realm-specific. Switch to the correct realm.
-

7. Sessions

What it is

Shows **who is currently logged in** to the Keycloak realm. Lists every active user session.

What each column shows

Column	What it means
User	Who logged in
IP Address	Where they logged in from
Started	When the session began
Last access	Last activity time
Clients	Which apps they accessed

What we do here

- **View active sessions** — See who is currently using the system
- **Log out a specific user** — Select user → Sign out (immediate — no warning)
- **Log out ALL users** — Useful after making critical security changes

Common issues

- **"I changed a user's role but they still have old access"** — Their session token still has the old role. Log them out from here, they'll get the new role when they log back in.
 - **"Too many sessions"** — Users might not be logging out properly. Set shorter session timeouts.
-

8. Events

What it is

A log of everything that happens in Keycloak — logins, failures, admin actions.

Two types of events

Event Type	What it records	Example
Login events	User login attempts	Login success, login failure, logout
Admin events	Admin actions	Create user, update client, delete role

What we do here

- **Investigate login failures** — "Why couldn't Amina log in?" → Check events for error message
- **Audit admin actions** — "Who changed the client settings?"
- **Forward events** — Can send to an external logging system (advanced)

Event types you'll see

- **LOGIN** — Successful login
- **LOGIN_ERROR** — Failed login (hover over it to see why)
- **LOGOUT** — User logged out
- **REFRESH_TOKEN** — Token was refreshed
- **REGISTER** — User registered themselves (if self-registration is on)
- **UPDATE_PROFILE** — User changed their profile
- **DELETE_ACCOUNT** — User deleted their account

Common issues

- **Events show nothing** — Event logging might be disabled. Go to Events → Config to enable.
 - **"LOGIN_ERROR: invalid_user_credentials"** — Wrong username or password
 - **"LOGIN_ERROR: user_not_found"** — Username does not exist in Keycloak
-

9. Identity Providers

What it is

Identity providers let users log in using OTHER systems — like "Login with Google" or "Login with another government Keycloak".

What we see in our setup

In **govasset** realm:

```
User-defined:
  Keycloak OpenID Connect
  OpenID Connect v1.0
  SAML V2.0
```

In **master** realm:

```
User-defined:
  Keycloak OpenID Connect
  OpenID Connect v1.0
  SAML V2.0
```

When we would use this

- **Connect two Keycloak servers** — If one ministry runs their own Keycloak and we want users from that Keycloak to log into ours
- **Department-level SSO** — A ministry wants their staff to use their existing login system
- **Government ID integration** — If Tanzania deploys a national single sign-on system

What we do here

Currently: **Nothing**. We don't use identity providers. Our Keycloak is the source of truth.

Common issues

- **"Login loop"** — Incorrect identity provider configuration causes infinite redirects
- **"Email mismatch"** — The email from the identity provider doesn't match any Keycloak user

10. User Federation

What it is

User federation connects Keycloak to an external user database (like LDAP, Active Directory, or another database). Instead of storing users in Keycloak, it reads them from your existing system.

When we would use this

- A ministry already has an **Active Directory** (Windows domain) with all their employees
- We want to reuse those existing accounts instead of creating new ones in Keycloak
- The user logs in with their existing Windows/network password

What we do here

Currently: **Nothing**. We create users directly in Keycloak.

Common issues

- **"Can't connect to LDAP"** — Wrong server address, port, or credentials
- **"Users not syncing"** — The sync timer hasn't triggered or connection is down
- **"Passwords not working"** — LDAP password policies might differ from Keycloak's

11. Authentication

What it is

The **authentication** section controls HOW users log in. It defines the login flow — the steps between "user clicks login" and "user is authenticated".

Key concepts

Flows — A flow is a sequence of steps:

- **Browser flow** — What happens when user logs in via a web browser
- **Direct grant flow** — What happens when an app sends username + password directly (Type B)
- **Registration flow** — What happens when a user signs up (if self-registration is enabled)
- **Reset credentials flow** — What happens during forgot password

Each flow has execution steps:

Example – Browser Flow:

Step 1: Cookie – checks if user already has a Keycloak session cookie

Step 2: OTP Form – asks for 2FA code (if required)

Step 3: Username/Password Form – shows the login form

Step 4: etc.

Bindings — Which flow is used for which scenario:

- **Browser flow** → **browser**
- **Direct grant flow** → **direct grant**
- **Reset credentials flow** → **reset credentials**
- **Registration flow** → **registration**

What we do here

The most common things we change:

A) Required Actions: These are things users MUST do after their next login:

- **UPDATE_PASSWORD** — Force password change
- **CONFIGURE_TOTP** — Set up 2-factor authentication
- **VERIFY_EMAIL** — Verify email address
- **UPDATE_PROFILE** — Update profile information

B) Password Policy: Rules for password strength:

- Minimum length
- Require digits, special characters, uppercase/lowercase
- Not equal to username
- Password history (can't reuse last X passwords)

C) OTP Policy: 2-factor authentication settings:

- Type: TOTP (time-based) or HOTP (counter-based)
- Digits: 6 (standard)
- Look-ahead window: How many codes forward/backward are accepted

Common issues

- **"User is stuck in a loop"** — Required actions keep redirecting. Go to User → Required actions and clear them all.
- **"Password too simple"** — The password policy is rejecting it. Either make the password stronger or relax the policy.
- **"2FA code not working"** — Time sync issue. Ensure Keycloak server time matches the authenticator app time.
- **Login flow is broken** — Someone changed the default flow. Reset to default.

12. Security Defenses

What it is

Settings that protect Keycloak from attacks like brute force logins.

Key settings

Brute Force Detection:

Setting	What it does
Enabled	ON = track failed logins
Max login failures	Number of failures before lockout (we use 5)
Wait increment	How long to wait before allowing next attempt (we use 15 minutes)
Quick login check	Milliseconds to wait between login attempts (slows down automated attacks)
Permanent lockout	ON = account stays locked until admin intervenes

Headers:

- Controls security-related HTTP headers Keycloak sends (X-Frame-Options, etc.)

What we do here

- **Enable brute force detection** for the govasset realm (important!)
- **Unlock a user** who was locked out (set Failure count to 0)
- **Disable temporarily** if testing and keep getting locked out

Common issues

- **"I keep getting locked out while testing"** — Disable brute force detection temporarily, or set Max failures higher (e.g., 100).
 - **"User locked out and no admin available"** — Wait the wait increment time, or ask admin to reset failure count.
 - **Settings not saving** — You might need admin role in the correct realm.
-

13. Server Info

Where it is

A gear icon  at the bottom of the menu in new Keycloak, or at the bottom of some sidebars.

What it shows

Section	What it tells you
Info	Keycloak version (e.g., 26.1.5), server time, uptime
Database	Which database is being used (PostgreSQL or H2)
Features	What features are installed (jdbc-postgresql, cdi, etc.)
Memory	Total memory, free memory, used memory
Profiles	Which Keycloak features are enabled
Environment	Environment variables Keycloak can see

What we do here

- **Check version** before upgrading
- **Verify PostgreSQL connection** (Db tab → Vendor: postgresql)
- **Check memory usage** if Keycloak is running slow
- **Check environment variables** if we aren't sure our env vars are being picked up

Common issues

- **"Server Info shows H2 database"** — `KC_DB` env var is not set correctly
 - **"Java heap out of memory"** — Memory usage is too high. Adjust `JAVA_OPTS_APPEND`.
 - **"KC_DB_URL not found"** — The environment variable is not being passed to the container
-

14. Realms (Advanced)

Where it is

Usually under the realm dropdown → "Create realm" or in the sidebar of the master realm.

What it does

Creates a new, completely separate realm. Each realm has its own:

- Users
- Clients
- Roles
- Authentication flows
- Sessions
- Database (can be separate from other realms)

When you would create a new realm

- A new ministry wants COMPLETE isolation (their own user database, their own Keycloak settings)
- Testing — create a dev realm to experiment without affecting production

What we do here

Currently: **Nothing**. We use one single `govasset` realm.

15. Quick Reference — Common Tasks

Creating an external system (Group 2, etc.)

1. Ensure you're in `govasset` realm (not master)
2. Clients → Create client
3. Set Client ID (e.g., "group2-app")
4. Client authentication: ON
5. Standard flow: ON
6. Direct access grants: ON (if they need Type B login)
7. Save
8. Go to Settings → Valid redirect URIs:
 - Add their callback URL (e.g., `https://group2.com/auth/callback`)
9. Add Valid post logout redirect URIs:
 - Add their logout URL
10. Go to Credentials → Copy Client Secret
11. Send them: Client ID, Client Secret, our Keycloak URL, our API URL

Creating a new user

1. Ensure you're in `govasset` realm
2. Users → Add user
3. Username: (username), Email: (optional)
4. Save
5. Credentials → Set password → Enter password
6. Temporary: OFF (or ON if they should change it)
7. Role mapping → Assign role → Select role (e.g., "user")

Unlocking a locked-out user

1. Users → Find the user → Click them
2. Go to "Security defenses" tab → Set "Failure count" to 0
3. OR: Go to Security Defenses in sidebar and use the Unlock all users button

Forcing a user to change password

1. Users → Find user → Click them
2. Required user actions → Select "Update Password" → Add
3. Next time they log in, they'll be forced to change password

Investigating why login failed

1. Events → Login events
2. Find the failed attempt (LOGIN_ERROR)
3. Hover over or click it to see the error message
4. Common errors:
 - "invalid_user_credentials" → wrong password
 - "user_not_found" → username doesn't exist
 - "user_disabled" → account is disabled
 - "invalid_input" → malformed request

16. Troubleshooting Common Issues

Symptom	Most likely cause	Fix
"Invalid redirect URI"	Callback URL in login link doesn't match Valid Redirect URIs in client config	Check for trailing slash, http vs https
403 on account page	Web Origins empty in account-console client	Add https://your-domain to Web Origins
User can log in but gets "User not found"	User exists in Keycloak but not in Django	Create user in Django admin or run <code>setup_demo_data</code>
Admin console says "temporary admin"	Using bootstrapped admin from env vars	Create a real admin user and delete the temporary one
"Client not found"	Client ID in login URL doesn't match any client in this realm	Check realm is correct (govasset, not master)
Login page shows but hangs after login	Wrong Keycloak URL or port	Check <code>KEYCLOAK_SERVER_URL</code> in Django settings

Symptom	Most likely cause	Fix
Changes not taking effect	Session still has old data	Log user out from Sessions tab, or wait for token expiry
"Access denied" in admin console	User doesn't have admin role in master realm	Assign admin role in master realm, not govasset
Keycloak won't start on Railway	Out of memory	Add/check JAVA_OPTS_APPEND=-Xmx256m -Xms128m
Manage Account page errors	Account console client misconfigured	Check Web Origins and Valid Redirect URIs

Summary — The Mental Model

Think of Keycloak as a **security guard building**:

Sidebar item	Real-world analogy
Realm	A separate building (master = admin building, govasset = user building)
Clients	The doors into the building — each system (Django, Group 2) has its own door
Client Scopes	What each door allows through (ID card only vs ID + badge)
Realm Roles	Types of badges: employee, manager, security
Users	People with ID cards
Groups	Departments — assign badges to whole teams at once
Sessions	Who is currently inside the building
Events	Security camera footage — who came in, who was turned away
Identity Providers	Other buildings' ID cards we also accept
User Federation	A shared employee directory from another department
Authentication	The security checkpoints: "Show ID, scan fingerprint, etc."
Security Defenses	Lockdown rules: "3 failed badge scans = security called"
Server Info	The building maintenance panel — power, cooling, version info